

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ
І.І.МЕЧНИКОВА

Факультет математики, фізики та інформаційних технологій

Кафедра математичного забезпечення комп'ютерних систем

ЗВІТ

з освітнього компоненту «Програмування»

Лабораторна робота №4

на тему: СИСТЕМА УПРАВЛІННЯ ПЕРСОНАЛОМ (ІЄРАРХІЯ
КЛАСІВ PERSON)

Студента 1 курсу, денна форма навчання

спеціальність F7 «Комп'ютерна інженерія»

Цуркана Максима Сергійовича

Керівник: к.т.н., доц. Шестопапов С.В.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

ОДЕСА – 2026

ЗМІСТ

ВСТУП	3
ЗАВДАННЯ	4
1. ТЕОРЕТИЧНА ЧАСТИНА	5
1.1 Поняття ООП та успадкування	5
1.2 Опис об'єктної частини завдання	6
2. ПРАКТИЧНА ЧАСТИНА	7
2.1 Опис класів та ієрархії	7
2.2 Інтерфейс програми	12
ВИСНОВКИ	16
СПИСОК ЛІТЕРАТУРИ	17

ВСТУП

Мета роботи – реалізувати клас `Person` та клас(и)-спадкоємці відповідно до варіанта завдання, а також програмну систему для роботи з екземплярами цих класів.

Під час виконання роботи необхідно використовувати основні принципи об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Згідно із завданням потрібно взяти готовий клас `Person`, реалізувати два класи-спадкоємці з перевизначенням оператора вставки в потік (або методу `to_string()`, або методу `print()`), а також побудувати програмну систему з можливістю додавання, видалення та виведення інформації про екземпляри типу `Person` та його спадкоємців через текстове меню. Для зберігання об'єктів використовується вектор вказівників на `Person`.

ЗАВДАННЯ

1. Взяти клас Person із четвертої лабораторної (див. таблицю 1).
2. Реалізувати два спадкоємці класу Person (див. таблицю 2 за варіантами).
У спадкоємцях перевизначити оператор вставки в потік або, відповідно, або метод `to_string()`, або метод `print()`.
3. Створити програмну систему з можливістю додавання, видалення, виведення інформації про екземпляри типу Person та його спадкоємців (додатково: пошук). Використовувати вектор вказівників на Person (як менш оптимальний варіант припустимо використовувати кілька векторів елементів відповідних класів).

Номер варіанта – це номер студента у списку групи (тобто в журналі).

В основній програмі бажано реалізувати гнучкий сценарій роботи програми, наприклад, через текстове меню, щоб користувач міг вибирати свої чергові дії у довільному порядку.

Лабораторна 4: Розробити клас Person, який описує людину, з наступними можливостями:

- Полями та методами зазначеними у таблиці (за варіантами).
- Створити конструктор без параметрів та конструктор з параметрами.
- Створити метод(и) «змінити e-mail» та/або «змінити номер телефону» (за наявності відповідних полів у завданні).
- При створенні людини (у конструкторі) перевіряти зріст та вагу (за наявності відповідних полів у завданні).
- Реалізувати вставку людини у потік виведення (роздруківку), або метод `to_string()`, або метод `print()`.

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП та успадкування

Об'єктно-орієнтоване програмування (ООП) – це метод програмування, заснований на поданні програми у вигляді сукупності взаємодіючих об'єктів, кожен з яких є екземпляром певного класу. Класи утворюють ієрархії спадкування, що дозволяють будувати складні системи з компонентів, які повторно використовуються [1].

Основними принципами ООП є [2, 3]:

- Інкапсуляція – механізм, який поєднує дані та методи, що обробляють ці дані, і захищає їх від зовнішнього некоректного впливу. В C++ реалізується за допомогою специфікаторів доступу `private`, `protected` та `public`.
- Наслідування – механізм, за допомогою якого новий клас (спадкоємець) набуває властивостей та поведінки існуючого класу (базового). У даній роботі `Worker` та `Electrician` успадковують від `Person`.
- Поліморфізм – здатність об'єктів різних класів відповідати на одне й те ж саме повідомлення кожен по-своєму. Реалізується через віртуальні методи (`virtual`) та перевизначення (`override`).

Ключовим інструментом для реалізації динамічного поліморфізму в C++ є ключове слово `virtual`. Якщо метод оголошено як `virtual` у базовому класі, то при виклику через вказівник або посилання на базовий клас буде викликано реалізацію з фактичного класу об'єкта.

Перевизначення оператора вставки в потік (`operator<<`) є зручним механізмом для виведення об'єктів у стандартний вивід. В даній роботі кожен клас перевизначає `operator<<` та метод `to_string()`, що повертає рядкове представлення об'єкта.

1.2 Опис об'єктної частини завдання

Система управління персоналом будується навколо базового класу `Person`, який містить загальні атрибути особи: ПІБ, зріст, вагу, стать, країну проживання та електронну пошту. Клас надає два основні методи-предикати: перевірку зросту понад 200 см та перевірку проживання в Україні.

Клас `Worker` (Робітник) розширює `Person` додатковими полями, що описують трудову діяльність: завод, посада та оклад. Метод `increaseSalary()` реалізує підвищення окладу залежно від посади – від 30% для директора до 5% для інших посад. Метод `changePosition()` дозволяє змінити посаду.

Клас `Electrician` (Електрик) також є спадкоємцем `Person` і зберігає стаж роботи та розряд. Через вказівник зберігається поточний об'єкт (підстанція, агрегат тощо), на якому електрик працює. Допоміжний клас `Object` (не є спадкоємцем `Person`) зберігає назву об'єкта та мінімальні вимоги до стажу й розряду для роботи. Метод `canWorkOnObject()` перевіряє відповідність кваліфікації, метод `changeObject()` змінює призначення з перевіркою допуску.

Клас `PersonSystem` реалізує сховище об'єктів у вигляді вектора унікальних вказівників `std::unique_ptr<Person>`. Це дозволяє зберігати об'єкти різних типів (`Person`, `Worker`, `Electrician`) та звертатися до них поліморфно. Метод `printElectriciansForObject()` виводить усіх електриків, що відповідають вимогам конкретного об'єкта.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів та ієрархії

Нижче наведено структуру всіх класів проекту.

1) Клас «Person» (базовий):

- Захищені поля: `fullName` (ПІБ), `height` (зріст), `weight` (вага), `gender` (стать), `country` (країна), `email` (ел. пошта).
- Конструктори: без параметрів, з параметрами, копіювання.
- Методи: `isTallerThan200()` – повертає `true` якщо зріст > 200 см; `livesInUkraine()` – повертає `true` якщо країна «Україна».
- Перевизначення `to_string()` та `operator<<` – виводять інформацію про об'єкт.
- Перевантажені оператори: `operator=` (присвоєння), `operator==` (рівність за ПІБ та email).

2) Клас «Worker» (спадкоємець Person):

- Захищені поля: `factory` (завод), `position` (посада), `salary` (оклад).
- Конструктори: без параметрів, з параметрами, копіювання.
- Метод `increaseSalary()` – збільшує оклад залежно від посади (директор +30%, менеджер +20%, інженер +15%, технік +10%, інші +5%).
- Метод `changePosition()` – змінює посаду.
- Перевизначення `to_string()` та `operator<<`.

3) Допоміжний клас «Object»:

- Приватні поля: `name` (назва об'єкта), `minExperience` (мін. стаж), `minGrade` (мін. розряд).
- Конструктори: без параметрів, з параметрами, копіювання.
- Повний набір гетерів та сетерів. Метод `to_string()` та `operator<<`.

4) Клас «Electrician» (спадкоємець Person):

- Приватні поля: experience (стаж, роки), grade (розряд), currentObject (вказівник на Object).
- Конструктори: без параметрів, з параметрами (необов'язковий параметр obj=nullptr), копіювання.
- Метод canWorkOnObject() (без параметрів) – перевіряє допуск до поточного об'єкта.
- Метод canWorkOnObject(const Object&) – перевіряє допуск до довільного об'єкта.
- Метод changeObject(Object*) – призначає новий об'єкт з перевіркою допуску, повертає bool.
- Перевизначення to_string() та operator<<.

5) Клас «PersonSystem»:

- Приватні поля: people – вектор std::unique_ptr<Person>; objects – вектор Object.
- Методи додавання: addPerson(), addWorker(), addElectrician().
- Методи видалення: removeByName(), removeByIndex().
- Методи виведення: printAll(), printWorkers(), printElectricians().
- Методи пошуку: findByName(), findIndex(), findByType().
- Методи для об'єктів: addObject(), findObject(), printObjects(), printElectriciansForObject().

2.2 Інтерфейс програми

Програма реалізує інтерактивне текстове меню, яке дозволяє користувачу обирати операції у довільному порядку.

Головне меню програми містить 16 пунктів: додавання Person/Worker/Electrician, видалення за іменем та за індексом, виведення всіх записів та записів за типом, пошук за іменем, операції над робітниками (збільшення окладу, зміна посади), операції над електриками (перевірка допуску, зміна об'єкта), а також робота з довідником об'єктів (додавання, виведення, пошук електриків для об'єкта).

При виборі пунктів додавання користувачу пропонується послідовно ввести всі поля нового об'єкта. Для Worker та Electrician спочатку вводяться поля базового класу Person, а потім додаткові поля спадкоємця.

Видалення можливе за точним ПІБ або за порядковим номером у списку (індексація з 0). Виведення записів використовує поліморфний метод print(), що викликає перевизначену версію to_string() відповідного класу.

Пошук здійснюється за повним ПІБ та виводить детальну інформацію про знайдений запис. Окремі пункти меню призначені для операцій над робітниками (зміна окладу залежно від посади, зміна посади) та електриками (перевірка допуску до поточного об'єкта, зміна призначеного об'єкта з перевіркою відповідності вимогам).

Довідник об'єктів дозволяє додавати нові об'єкти, виводити їх список та шукати електриків, що відповідають вимогам конкретного об'єкта.

ВИСНОВКИ

У ході виконання лабораторної роботи було розроблено програму на мові C++ з використанням принципів об'єктно-орієнтованого програмування на тему «Система управління персоналом».

В процесі роботи була реалізована ієрархія класів: базовий клас `Person`, обов'язковий спадкоємець `Worker` та додатковий спадкоємець `Electrician`. Додатково розроблено допоміжний клас `Object`, що не входить до ієрархії `Person` і використовується для зберігання вимог до кваліфікації електриків.

При розробці програми були застосовані всі основні принципи ООП:

- інкапсуляція – поля класів оголошено як `protected` або `private`, доступ здійснюється через гетери/сетери;
- наслідування – `Worker` та `Electrician` успадковують усі поля та методи `Person`, розширюючи їх;
- поліморфізм – метод `print()` оголошено як `virtual`, перевизначено в кожному класі; вектор вказівників на `Person` дозволяє поліморфно обробляти об'єкти різних типів.

Для зберігання колекції об'єктів використовується вектор `std::vector<std::unique_ptr<Person>>`, що забезпечує автоматичне управління пам'яттю. Реалізовано повний набір операцій: додавання, видалення за ім'ям та індексом, виведення всіх або відфільтрованих записів, пошук, а також специфічні операції над спадкоємцями.

У результаті виконання лабораторної роботи були закріплені практичні навички програмування на C++, роботи з класами, спадкуванням, поліморфізмом, колекціями, вказівниками та консольним інтерфейсом.

СПИСОК ЛІТЕРАТУРИ

4. Web Library, Основні поняття ООП. [Електронне видання] / Тернопіль 2026. – Режим доступу: <http://weblib.com.ua/blog/osnovni-ponyattya-oor>
5. W3School, C++ OOP (Object-Oriented Programming). Режим доступу: https://www.w3schools.com/cpp/cpp_oor.asp
6. Є.О. Віктор, О.А. Геренко, І.М. Шпінарева, Практикум з курсу Об'єктно-орієнтоване програмування мовою С#. [Електронне видання] / Одеса, 2026. – Режим доступу: <https://oor-practicum-csharp851f60.gitlab.io/>
7. Репозиторій GitHub лабораторної роботи. – Режим доступу: https://github.com/Offiks/lab4_PersonSystem